

(II) Digital Communication

Part A – Line Coding

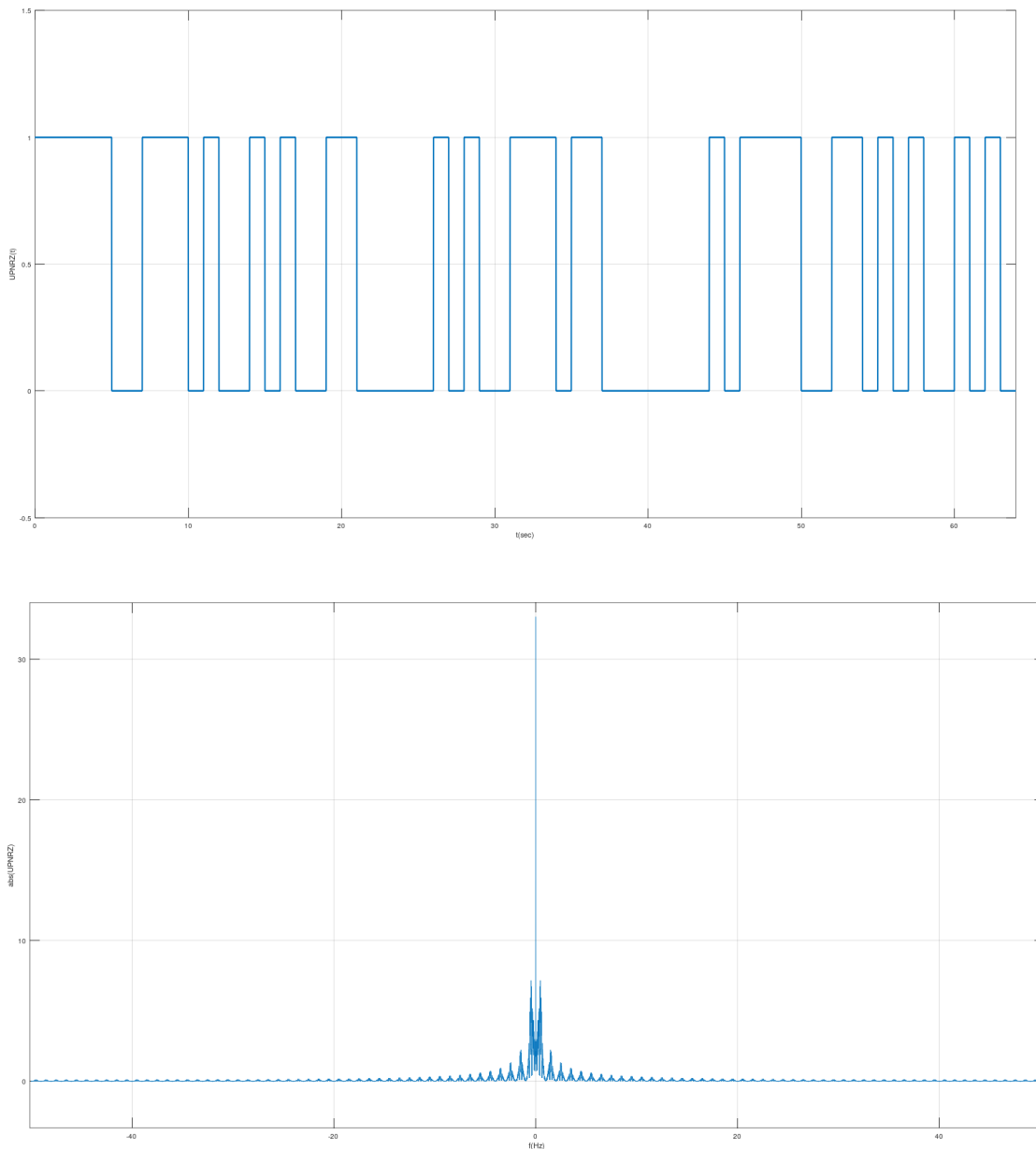
- unipolar non-return to zero octave code

```
1 % Parameters
2 Nb = 64; % Number of bits
3 Tb = 1; % Bit duration
4 fs = 100; % Sampling frequency
5 ts = 1/fs; % Sampling time
6 N = 6400; % Number of samples
7 df = 1/(N*ts); % Frequency resolution
8
9 % Generate a random binary sequence
10 b = rand(1, Nb) > 0.5;
11
12 % Time vector
13 t = 0:ts:Nb*Tb-ts;
14
15 % Unipolar NRZ signal generation
16 UPNRZ = [];
17 for i = 1:Nb
18     if b(i) == 1
19         UPNRZ = [UPNRZ ones(1, fs)];
20     else
21         UPNRZ = [UPNRZ zeros(1, fs)];
22     end
23 end
24
```

- Plotting the time and spectral domains of the pulses

```
25 % Plot Time Domain
26 figure(1)
27 plot(t, UPNRZ, 'LineWidth', 1.5); % plot the signal with time
28 axis([0 Nb*Tb -0.5 1.5]); % Set axis limits to show pulse levels clearly
29 xlabel('t(sec)');
30 ylabel('UPNRZ(t)');
31 grid on;
32
33
34 % Frequency Domain using fft
35 f = -0.5*fs : df : 0.5*fs-df ;
36 UPNRZ_f = fftshift(fft(UPNRZ))*ts; % non-periodic signal
37 figure(2)
38 plot(f,abs(UPNRZ_f));
39 xlabel('f(Hz)');
40 ylabel('abs(UPNRZ)');
41 grid on;
42
```

- temporal and spectral plot of unipolar non-return to zero respectively



Explanations of the temporal and spectral plots of Unipolar line code

As shown in the time domain graph, For Unipolar (Non-Return-to-Zero), Bit 1 represented by high voltage (+1 V) and Bit 0 represented by zero voltage (0 V). The voltage remains constant for the entire bit duration, so it is called "non-return-to-zero." Each Pulse corresponds to one of the 64 random bits.

The frequency-domain plot shows the magnitude of the FFT of the signal versus frequency. It is clear that there is a significant DC component and Most energy is concentrated at low frequencies, decreasing gradually at higher frequencies. These considered drawbacks of the unipolar non-return to zero as will be discussed later.

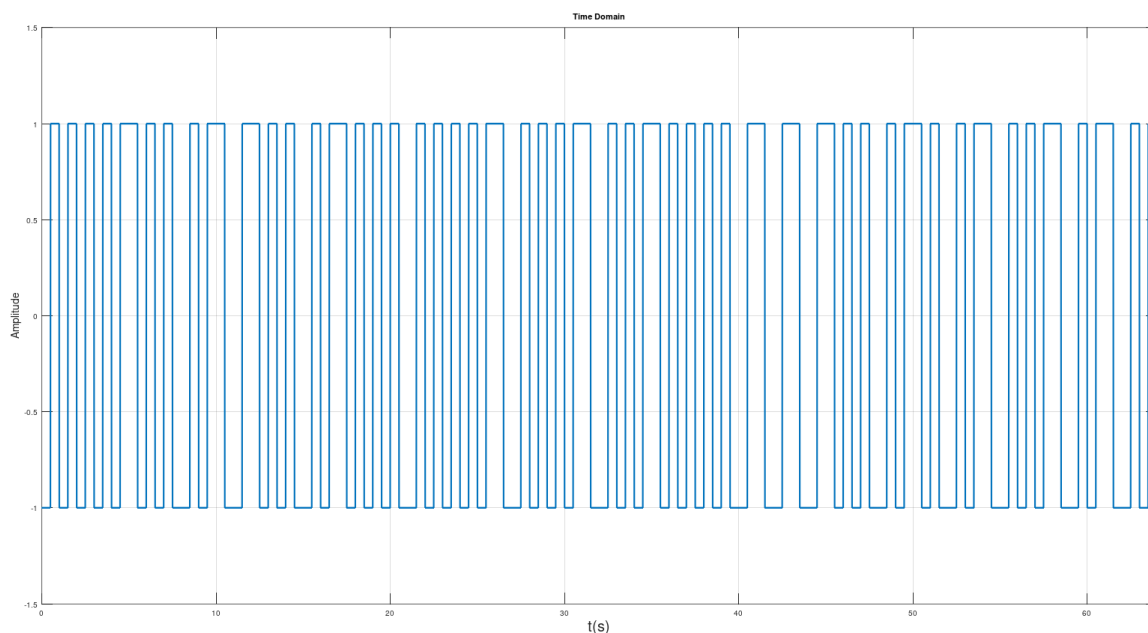
• Manchester Octave code

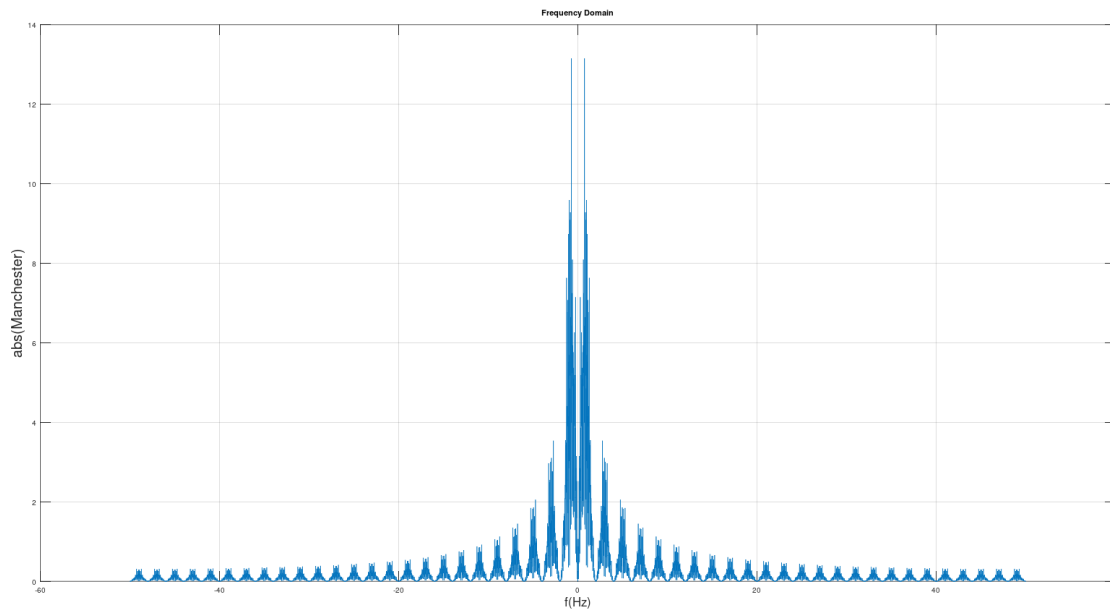
```
96 %%% Manchester Code %%%
97
98 %% Parameters
99 Nb = 64;           % Number of bits
100 Tb = 1;           % Bit duration
101 fs = 100;          % Sampling frequency
102 ts = 1/fs;         % Sampling period
103 N = 6400;          % Number of samples
104 df = 1/(N*ts);     % Frequency resolution
105
106 %% Generate a random binary sequence
107
108 b = rand(1, Nb) > 0.5;
109
110 %% Time vector for plotting
111 t = linspace(0, Nb*Tb, Nb*fs);           % Nb*fs is the number of samples
112
113
114 %% Manchester signal generation
115 Manchester = [];
116 for i = 1:Nb
117     if b(i) == 1
118         Manchester = [Manchester ones(1, fs/2) -ones(1, fs/2)]; %It starts 1 and ends -1 with half of the width of the pulse for each one
119     else
120         Manchester = [Manchester -ones(1, fs/2) ones(1, fs/2)]; %It starts -1 and ends 1 with half of the width of the pulse for each one
121     end
122 end
123
```

• Plotting the time and spectral domains of the pulses

```
125 %
126 %%Plotting Time Domain
127 figure;
128 plot(t, Manchester, 'LineWidth', 1.5);
129 title('Time Domain');
130 xlabel('t(s)', 'FontSize', 20);
131 ylabel('Amplitude', 'FontSize', 14);
132 axis([0 Nb*Tb -1.5 1.5]);           % Set axis limits to show pulse levels clearly
133 grid on;
134
135
136 % Frequency Domain using fft
137 f = -0.5*fs : df : 0.5*fs-df;
138 Manchester_f = fftshift(fft(Manchester))*ts; % non-periodic signal
139
140 figure(2)
141 plot(f, abs(Manchester_f));
142 title('Frequency Domain');
143 xlabel('f(Hz)', 'FontSize', 17);
144 ylabel('abs(Manchester)', 'FontSize', 20);
145 grid on;
146
```

• temporal and spectral plot of Manchester line code respectively



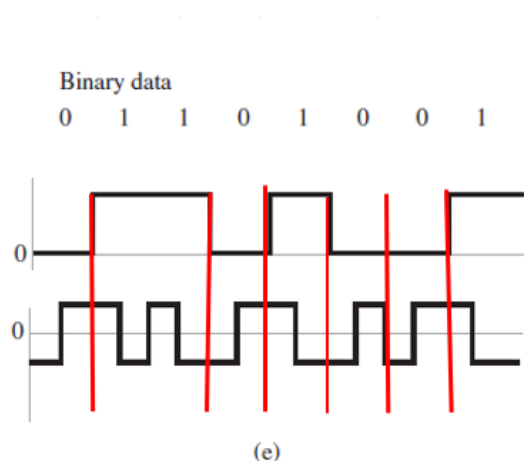


Explanations of the temporal and spectral plots of Manchester Line code

In this method of signaling, bit 1 is represented by a positive pulse followed by a negative pulse, with both pulses being of equal amplitude and half-symbol width. For bit 0, the polarities of these two pulses are reversed. The waveform alternates within each bit period according to the bit value, making the clock recovery easier. Compared to Unipolar NRZ, the Manchester signal switches more frequently.

In the frequency domain plot, The Manchester code suppresses the dc component and has relatively insignificant low-frequency components, regardless of the signal statistics.

Quick illustration of the Manchester code (below) compared to the normal on–off signaling (Up)



Comparison between Manchester and unipolar non-return to zero line codes

Unipolar non-return to zero-line code advantages:

- **Simple implementation**
- **Occupies less bandwidth**
- **Higher power efficiency due to fewer transitions**

Unipolar non-return to zero-line code disadvantages:

- **Has significant DC component (due to long sequences of 1s)**
- **Less immune to noise, as noise depends on the difference between levels of 1 and 0**
- **Poor self-synchronization**
- **No error detection or correction can be used**

Manchester line code advantages:

- **No DC component**
- **Excellent synchronization**

Manchester line code Disadvantages:

- **Requires double bandwidth compared to NRZ**
- **High power consumption due to frequent transitions**

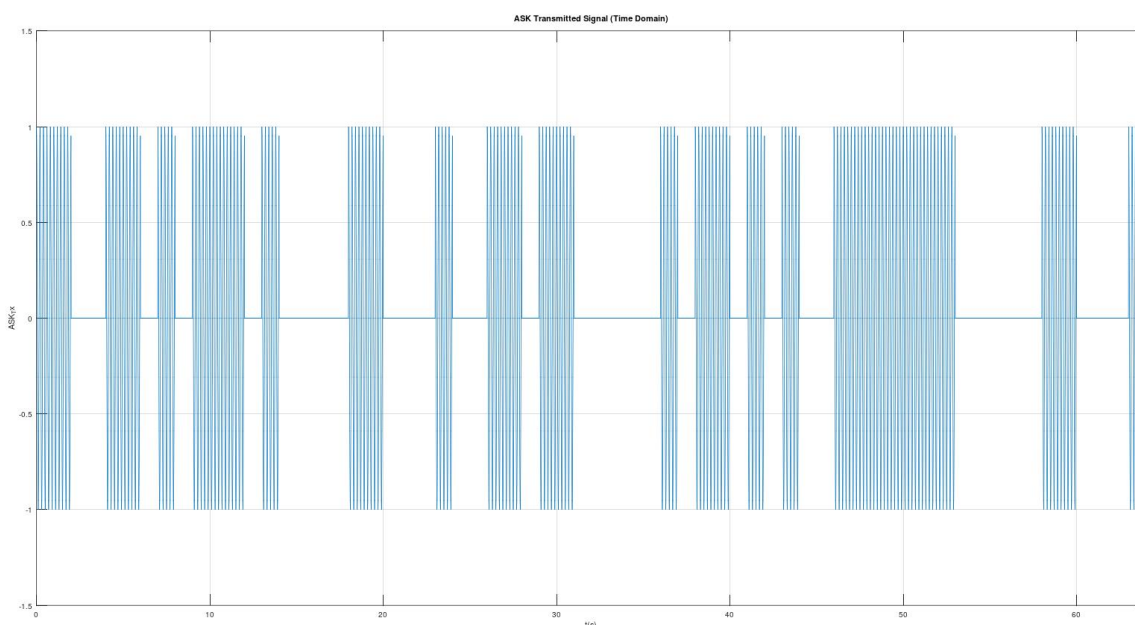
Part B – Digital Modulation

ASK Digital Modulation

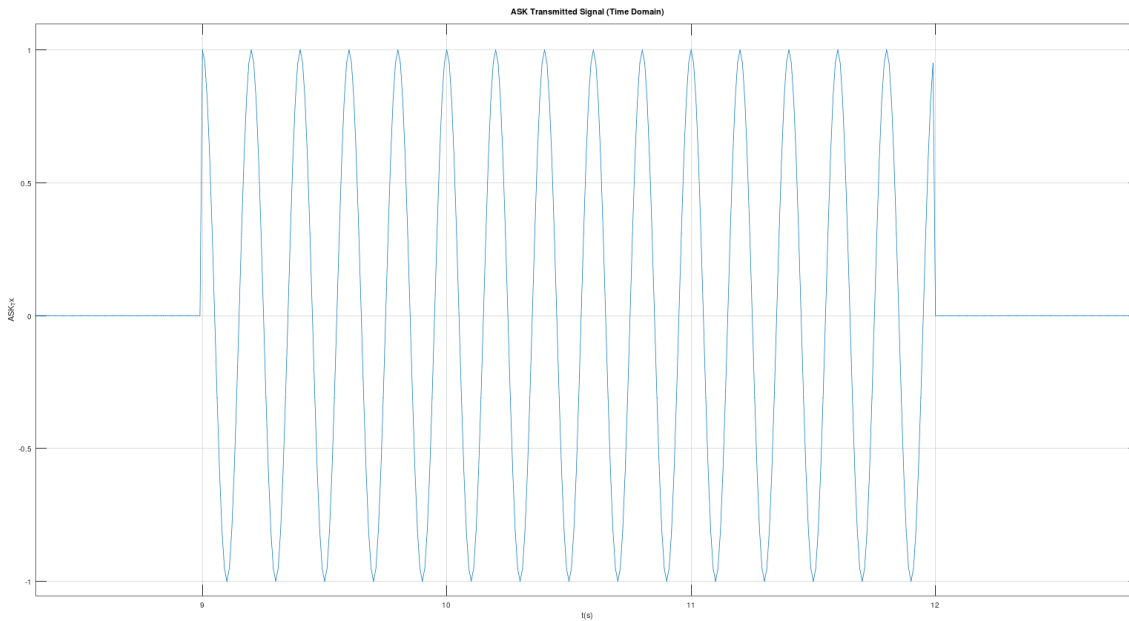
- Generating 64 random bits, Bit 0 = 0 and Bit 1 =1

```
1  %%% Part B - Digital Modulation %%%
2  %%%%%%%%% ASK %%%%%%%%%
3  N = 64;                % Number of bits
4  Tb = 1;                % Bit period
5  Rb = 1/Tb;             % Bit rate
6  fc = 5*Rb;             % Carrier frequency
7  fs = 100;             % Sampling frequency
8  ts = 1/fs ;           % Sampling Time
9  df = 1/(fs*N*ts)      % Frequency resolution
10
11 % Generate a random binary sequence
12 b = rand(1,N) > 0.5 ;
13
14 % Time Vector
15 t = linspace(0, N*Tb , N*fs) ;
16
17 % ASK Binary Data generation (Baseband signal)
18 ASK = [];
19 for i = 1:N
20     if b(i) == 1
21         ASK = [ASK ones(1,fs)] ;
22     else
23         ASK = [ASK zeros(1,fs)] ;
24     end
25 end
26
27 % ASK Transmitter
28 c = cos(2*pi*fc*t);    % Carrier Signal
29 ASK_Tx = ASK.*c ;
30
31
32
33
34
35
36
37
38
39
40 % Spectrum of Transmitted Signal
41 f = -0.5*fs : df : 0.5*fs-df ;
42 ASK_Tx_f = fftshift(fft(ASK_Tx))*ts;
```

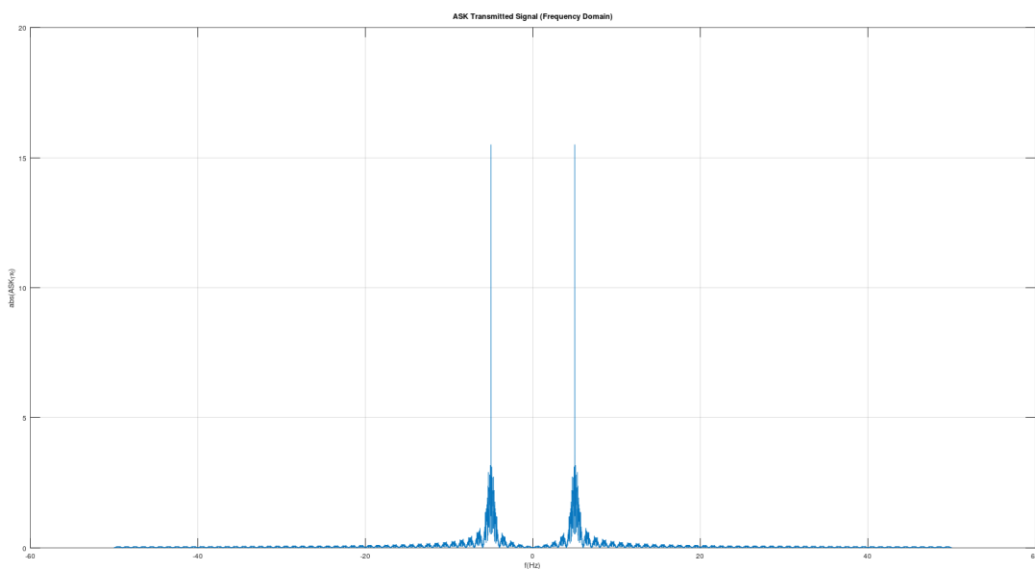
- The ASK Transmitted Signal (Time domain)



- **Zoomed Plot indicating that in one-bit period there are 5 cycles, because I stated in the beginning that the carrier frequency is 5 times the bit rate of the random binary data**



- **Spectrum of ASK transmitted signal**



- **The ASK Receiver with oscillator phase Code**

```

50 % Coherent ASK Receiver with Phase Error
51 C_Rx1 = cos(2*pi*fc*t + (pi/6));
52 C_Rx2 = cos(2*pi*fc*t + (pi/3));
53 C_Rx3 = cos(2*pi*fc*t + (pi/2));
54
55 ASK_Rx1 = ASK_Tx.*C_Rx1;
56 ASK_Rx2 = ASK_Tx.*C_Rx2;
57 ASK_Rx3 = ASK_Tx.*C_Rx3;
58
59 % Spectrum of Received Signal before the low-pass filter
60 ASK_Rx1_f = fftshift(fft(ASK_Rx1)) * ts;
61 ASK_Rx2_f = fftshift(fft(ASK_Rx2)) * ts;
62 ASK_Rx3_f = fftshift(fft(ASK_Rx3)) * ts;
63
64 %% Low-pass filter %%
65 H = abs(f) < 5;
66
67 % The output of the low pass filter (frequency domain)
68 ASK_Rx1_filtered_f = H.*ASK_Rx1_f ;
69 ASK_Rx2_filtered_f = H.*ASK_Rx2_f ;
70 ASK_Rx3_filtered_f = H.*ASK_Rx3_f ;
71
72 % The output of the low pass filter (Time domain)
73 ASK_Rx1_filtered_t = ifft(ifftshift(ASK_Rx1_filtered_f)) / ts;
74 ASK_Rx2_filtered_t = ifft(ifftshift(ASK_Rx2_filtered_f)) / ts;
75 ASK_Rx3_filtered_t = ifft(ifftshift(ASK_Rx3_filtered_f)) / ts;
76

```

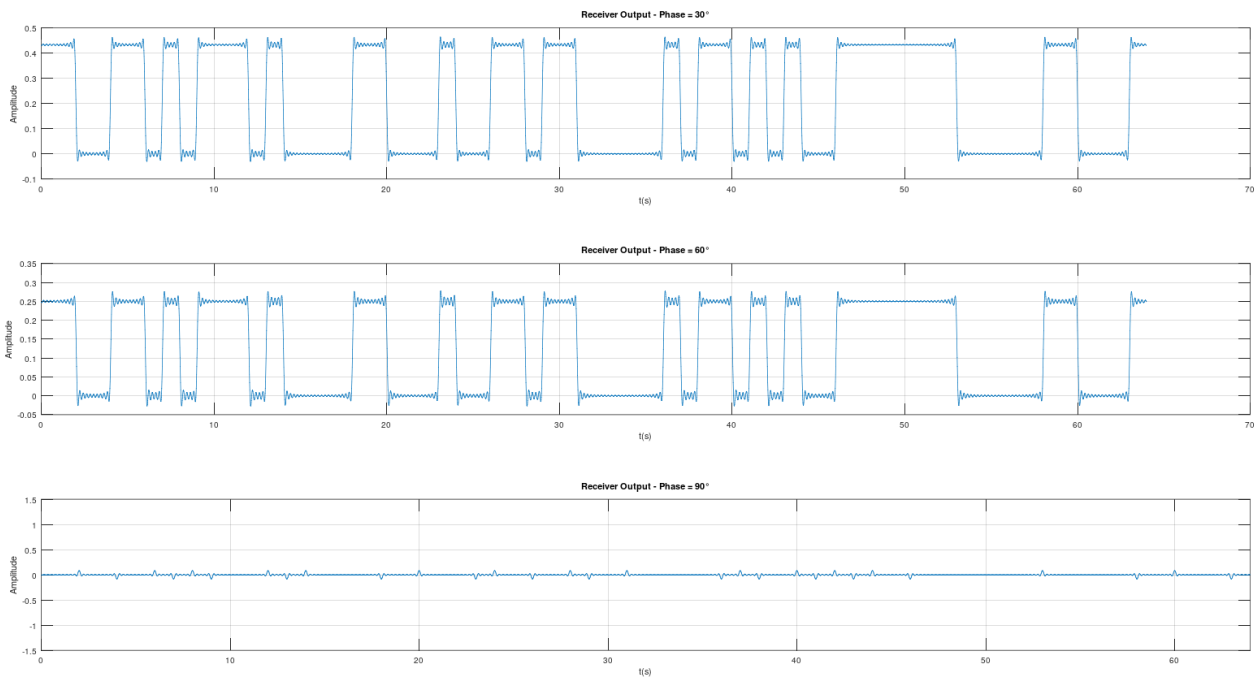
- The decision-making block code to retrieve the data

```

137 %%% Decision Making device %%%
138
139 decision_30 = zeros(1,N);
140 decision_60 = zeros(1,N);
141 decision_90 = zeros(1,N);
142
143 th = 0.2; % correct threshold to retrieve the data
144
145 for i = 1:N
146     idx = (i-1)*fs + 1 : i*fs;
147     % The average value of the low-pass filter output
148     z1 = mean(ASK_Rx1_filtered_t(idx));
149     z2 = mean(ASK_Rx2_filtered_t(idx));
150     z3 = mean(ASK_Rx3_filtered_t(idx));
151     decision_30(i) = z1 > th;
152     decision_60(i) = z2 > th;
153     decision_90(i) = z3 > th;
154 end
155

```

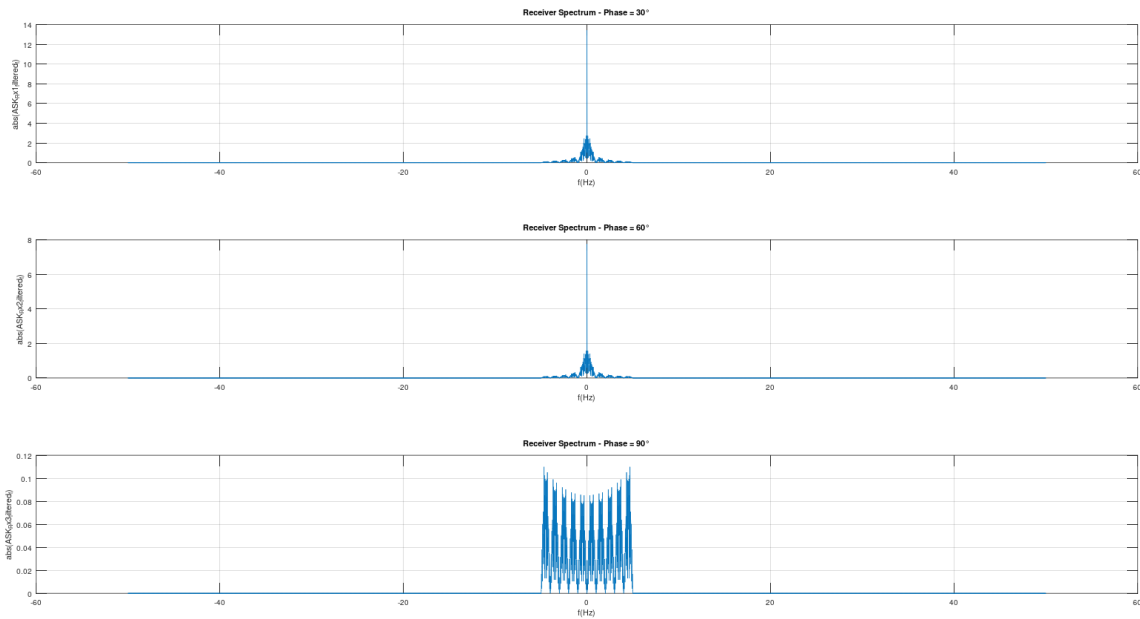
- The temporal plot of the output of the low-pass filter



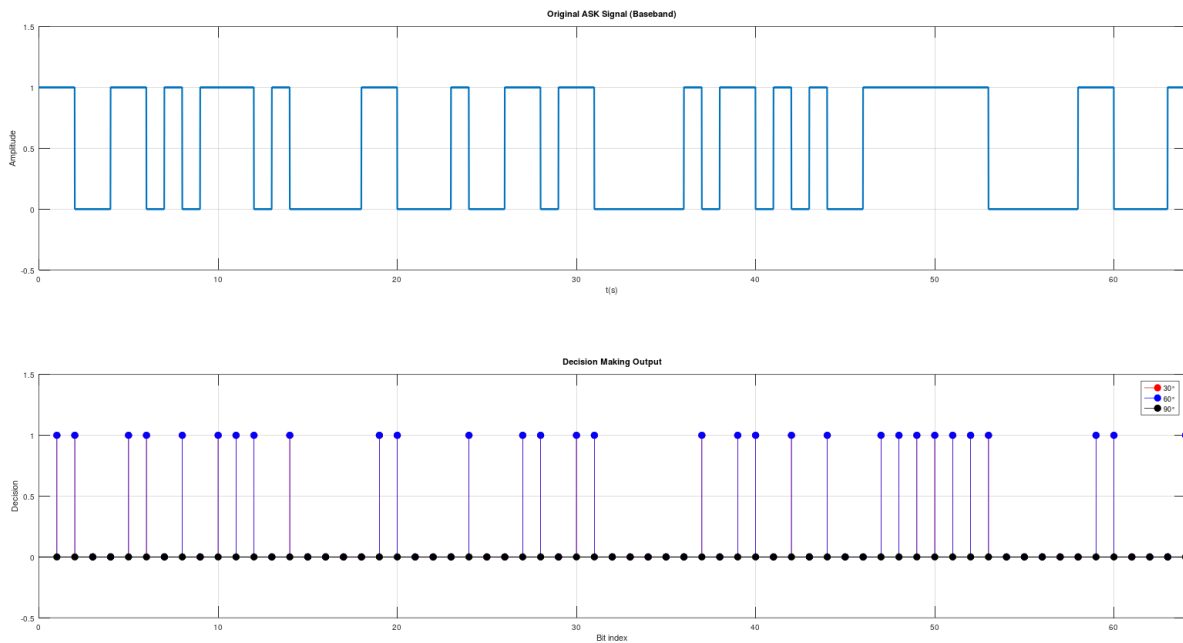
It is clear that when the phase is 30°, the amplitude is reduced to approximately 0.43, which is consistent with the assumption that the amplitude of the carrier oscillators is unity. The amplitude at the receiver = $0.5 \cos(\Phi)$.

Similarly, the amplitude when the phase is 60° is 0.25. And for the case of 90°, The receiver oscillator is orthogonal to the transmitted carrier making the amplitude is approximately zero.

- The Spectrum plot of the output of the low-pass filter



- Comparison between the output of the decision-making device and the original baseband signal



Notes and Comments

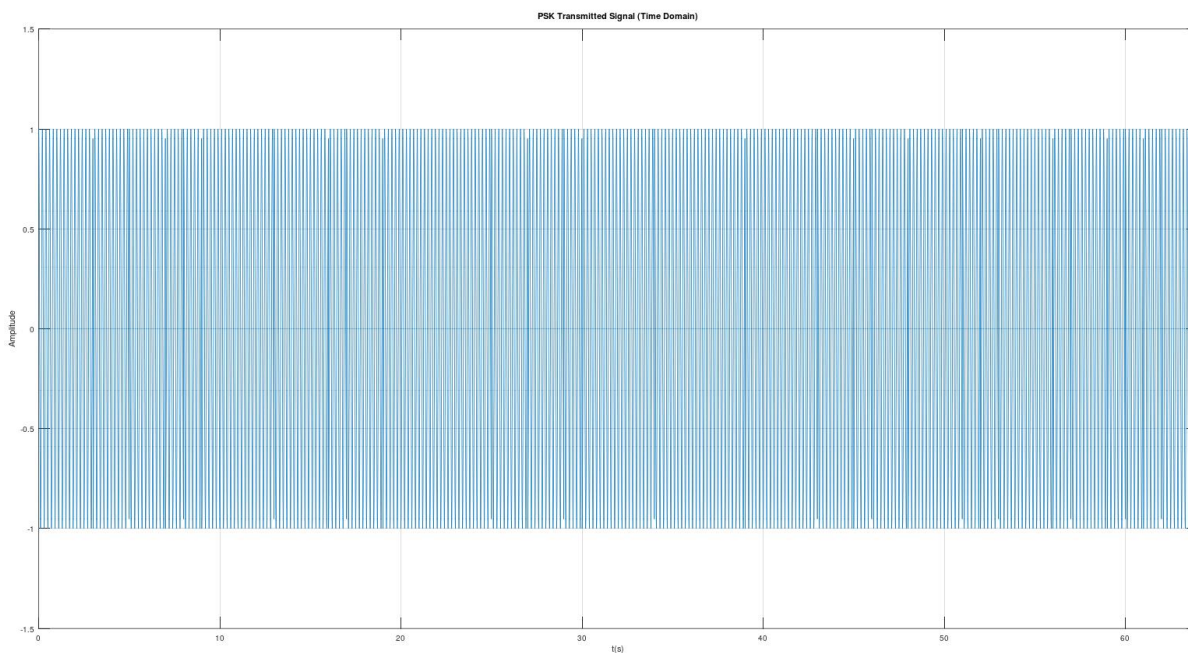
From the comparison, it is evident that the transmitted data can be successfully recovered by the decision-making device when an appropriate threshold is applied. However, when the receiver's oscillator experiences a phase shift of 90°, the output of the coherent detector is effectively zero, causing the decision-making process to fail completely and preventing accurate data retrieval.

PSK Digital Modulation

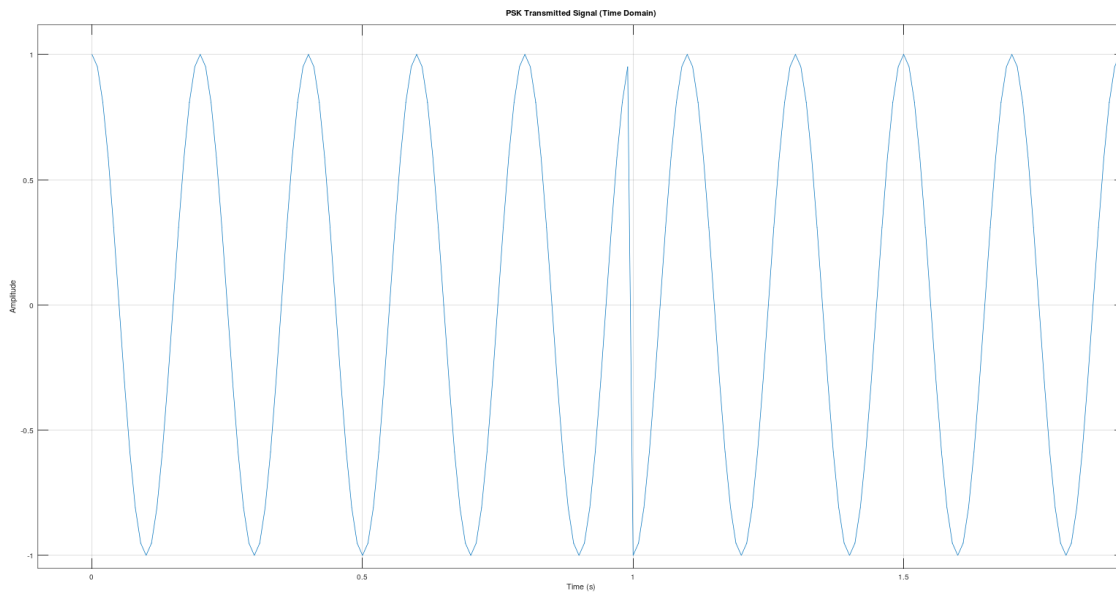
- Generating 64 random bits, Bit 0 = -1 and Bit 1 = 1 (Polar NRZ)

```
133 %%%%%% PSK %%%%%%
134
135 %% Parameters
136 N = 64;           % Number of bits
137 Tb = 1;           % Bit period
138 Rb = 1/Tb;        % Bit rate
139 fc = 5*Rb;        % Carrier frequency (> Rb)
140 fs = 100;         % Sampling frequency
141 ts = 1/fs;        % Sampling interval
142 df = 1/(N*Tb);    % Frequency resolution
143
144 %% Time vector
145 t = 0:ts:N*Tb-ts; % total points = N*fs
146
147 %% Generate random binary sequence
148 b = rand(1,N) > 0.5;
149
150 %% Baseband PSK signal generation
151 PSK = [];
152 for i = 1:N
153     if b(i) == 1
154         PSK = [PSK ones(1, fs)]; % bit 1 = 1
155     else
156         PSK = [PSK -ones(1, fs)]; % bit 0 = -1
157     end
158 end
159
161 %% PSK Transmitter
162 c = cos(2*pi*fc*t);
163 PSK_Tx = PSK.*c;
164
174 %% Frequency-domain plot (Transmitted)
175 f = -fs/2 : df : fs/2 - df;
176 PSK_Tx_f = fftshift(fft(PSK_Tx)) * ts;
```

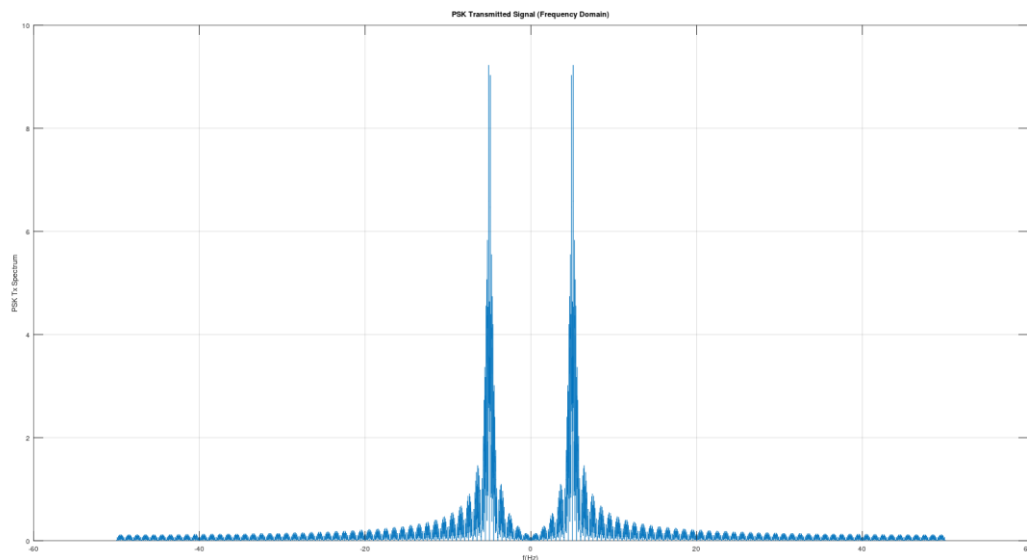
- The PSK Transmitted Signal (Time domain)



- **Zoomed Plot showing the phase shift 180° Of bit 0 after 1bit period**



- **Spectrum of PSK transmitted signal**



- **The PSK Receiver with oscillator phase Code**

```

228 %% Coherent PSK Receiver with Phase Errors %%
229 C_Rx1 = cos(2*pi*fc*t + (pi/6));
230 C_Rx2 = cos(2*pi*fc*t + (pi/3));
231 C_Rx3 = cos(2*pi*fc*t + (pi/2));
232
233 PSK_Rx1 = PSK_Tx.*C_Rx1;
234 PSK_Rx2 = PSK_Tx.*C_Rx2;
235 PSK_Rx3 = PSK_Tx.*C_Rx3;
236
237 % Spectrum of Received Signal before the low-pass filter
238 PSK_Rx1_f = fftshift(fft(PSK_Rx1)) * ts;
239 PSK_Rx2_f = fftshift(fft(PSK_Rx2)) * ts;
240 PSK_Rx3_f = fftshift(fft(PSK_Rx3)) * ts;
241
242 %%% Low-pass filter %%%
243 H = abs(f) < 5
244
245 % The output of the low pass filter (frequency domain)
246 PSK_Rx1_filtered_f = H.*PSK_Rx1_f ;
247 PSK_Rx2_filtered_f = H.*PSK_Rx2_f ;
248 PSK_Rx3_filtered_f = H.*PSK_Rx3_f ;
249
250 % The output of the low pass filter (Time domain)
251 PSK_Rx1_filtered_t = real(ifft(ifftshift(PSK_Rx1_filtered_f )) / ts);
252 PSK_Rx2_filtered_t = real(ifft(ifftshift(PSK_Rx2_filtered_f )) / ts);
253 PSK_Rx3_filtered_t = real(ifft(ifftshift(PSK_Rx3_filtered_f )) / ts);
254

```

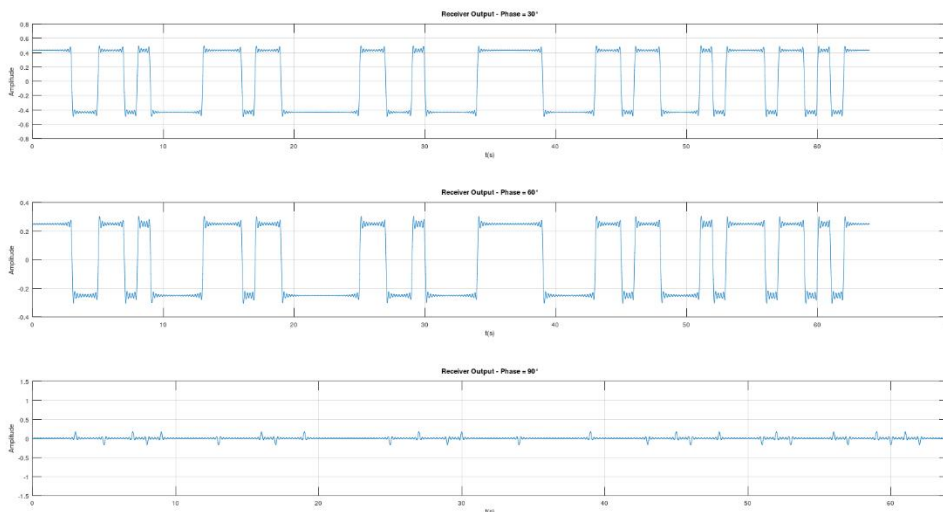
- The decision-making block code to retrieve the data

```

312 %%% Decision Making device %%%
313
314 decision_30 = zeros(1,N);
315 decision_60 = zeros(1,N);
316 decision_90 = zeros(1,N);
317
318 th = 0; % correct threshold to retrieve the data
319
320 for i = 1:N
321     idx = (i-1)*fs + 1 : i*fs;
322     % The average value of the low-pass filter output
323     z1 = mean(PSK_Rx1_filtered_t(idx));
324     z2 = mean(PSK_Rx2_filtered_t(idx));
325     z3 = mean(PSK_Rx3_filtered_t(idx));
326
327     % Decision making
328     if z1 > th
329         decision_30(i) = 1;
330     else
331         decision_30(i) = -1;
332     end
333
334     if z2 > th
335         decision_60(i) = 1;
336     else
337         decision_60(i) = -1;
338     end
339
340     if z3 > th
341         decision_90(i) = 1;
342     else
343         decision_90(i) = -1;
344     end
345 end

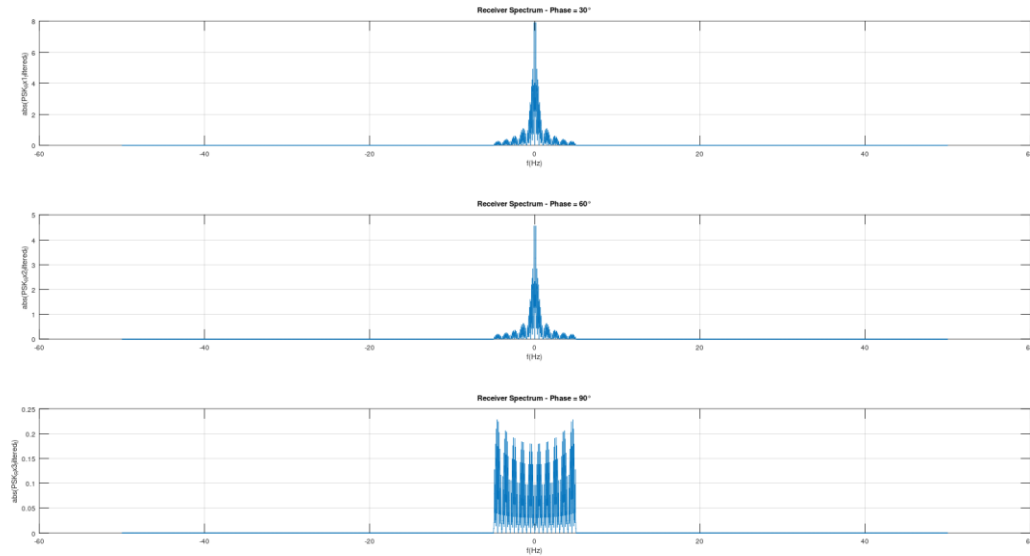
```

- The temporal plot of the output of the low-pass filter

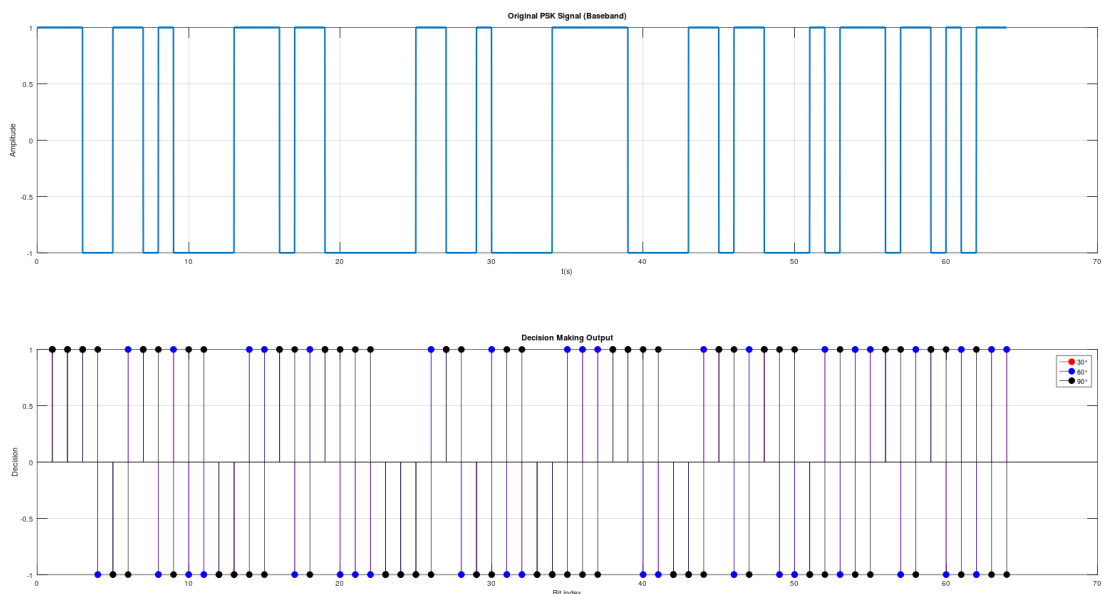


The amplitude reduction in PSK is the same as in ASK, except that the PSK swings from negative to positive

- The Spectrum plot of the output of the low-pass filter



- Comparison between the output of the decision-making device and the original baseband signal



Notes and Comments

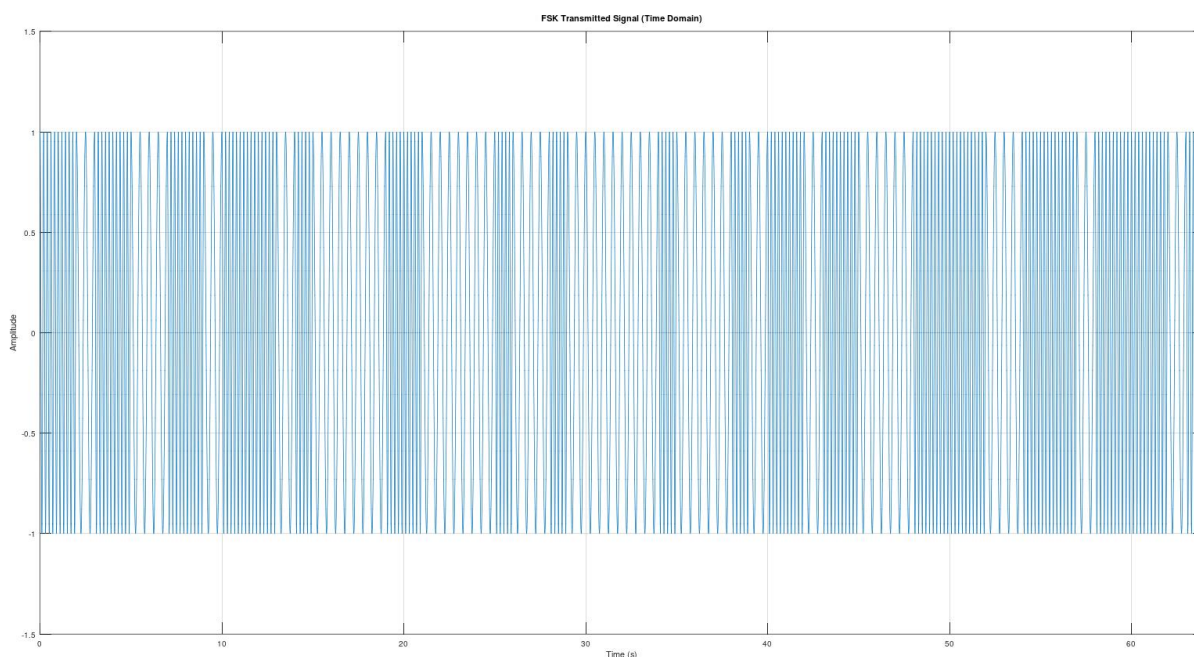
In PSK, the decision-making device recovers the transmitted data using a threshold set at 0, which allows it to distinguish between the two-phase states of the signal. This is different from ASK, where the threshold depends on the signal amplitude. As a result, PSK detection is less affected by amplitude variations, though significant phase shifts at the receiver can still impact performance.

FSK Digital Modulation

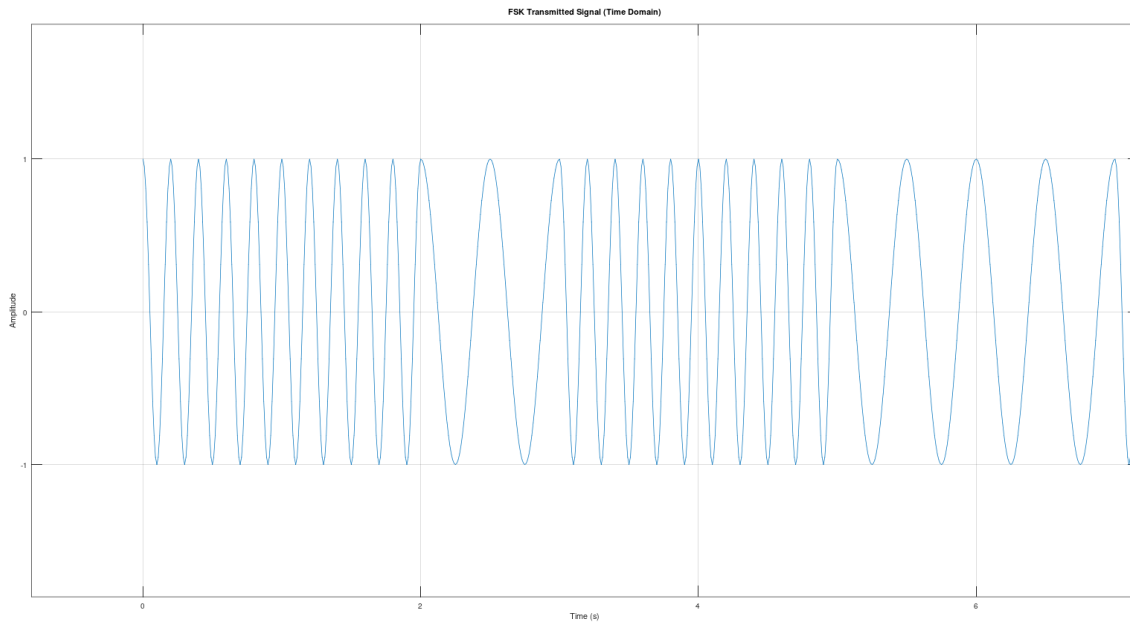
- Generating 64 random bits, Bit 0 = 0 and Bit 1 =1 (Unipolar NRZ)

```
266 %%%% FSK %%%
267
268 %% Parameters
269 N = 64; % Number of bits
270 Tb = 1; % Bit period
271 Rb = 1/Tb; % Bit rate
272 fs = 100; % Sampling frequency
273 ts = 1/fs; % Sampling interval
274 df = 1/(N*Tb); % Frequency resolution
275
276 f1 = 5*Rb; % Frequency for bit '1'
277 f0 = 2*Rb; % Frequency for bit '0'
278
279 %% Time vector
280 t = 0:ts:N*Tb-ts; % total points = N*fs
281
282 %% Generate random binary sequence
283 b = rand(1,N) > 0.5; % unipolar 0/1
284
285 %% Baseband FSK signal (for plotting)
286 FSK = [];
287 for i = 1:N
288     if b(i) == 1
289         FSK = [FSK ones(1,fs)];
290     else
291         FSK = [FSK zeros(1,fs)];
292     end
293 end
294
295 %% FSK Transmitter
296 FSK_Tx = [];
297 for i = 1:N
298     ti = (i-1)*Tb:ts:i*Tb-ts;
299     if b(i) == 1
300         FSK_Tx = [FSK_Tx cos(2*pi*f1*ti)];
301     else
302         FSK_Tx = [FSK_Tx cos(2*pi*f0*ti)];
303     end
304 end
305
```

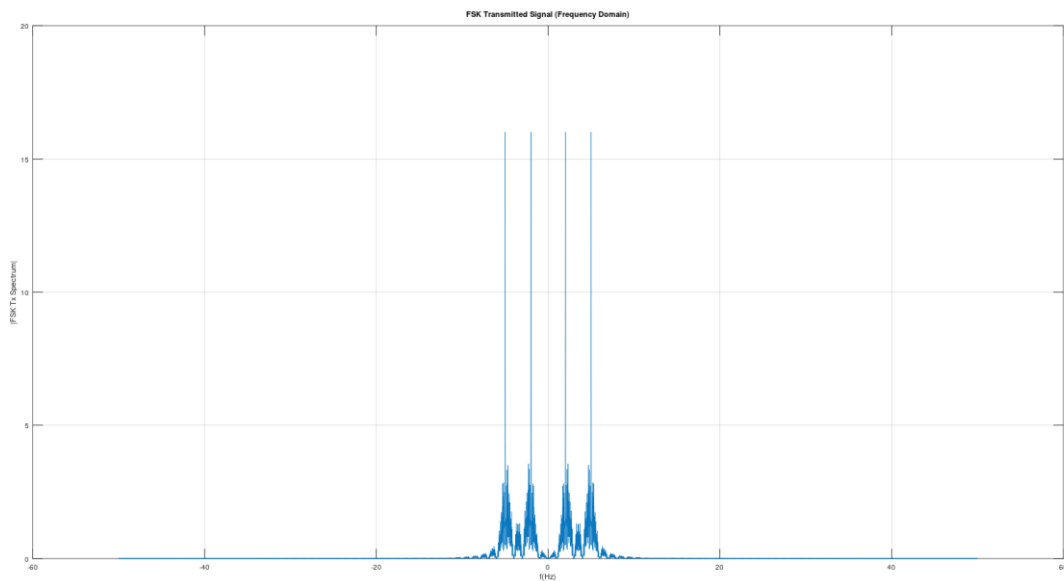
- The FSK Transmitted Signal (Time domain)



- **Zoomed Plot showing the different frequency carrier for bit 1 and 0 (bit 1 is 5 Hz and bit 0 is 2Hz)**



- **Spectrum of FSK transmitted signal**



- **The FSK Receiver with oscillator phase Code**

```

432 %% Coherent FSK Receiver with Phase Errors
433 C_Rx1 = cos(2*pi*f1*t + pi/6);
434 C_Rx2 = cos(2*pi*f1*t + pi/3);
435 C_Rx3 = cos(2*pi*f1*t + pi/2);
436
437 % Multiply by local oscillator
438 FSK_Rx1 = FSK_Tx .* C_Rx1 - FSK_Tx .* cos(2*pi*f0*t + pi/6);
439 FSK_Rx2 = FSK_Tx .* C_Rx2 - FSK_Tx .* cos(2*pi*f0*t + pi/3);
440 FSK_Rx3 = FSK_Tx .* C_Rx3 - FSK_Tx .* cos(2*pi*f0*t + pi/2);
441
442 % Spectrum of Received Signal before the low-pass filter
443 FSK_Rx1_f = fftshift(fft(FSK_Rx1)) * ts;
444 FSK_Rx2_f = fftshift(fft(FSK_Rx2)) * ts;
445 FSK_Rx3_f = fftshift(fft(FSK_Rx3)) * ts;
446
447 %%% Low-pass filter %%%
448 H = abs(f) < 2 ;
449
450 % The output of the low pass filter (frequency domain)
451 FSK_Rx1_filtered_f = H.*FSK_Rx1_f ;
452 FSK_Rx2_filtered_f = H.*FSK_Rx2_f ;
453 FSK_Rx3_filtered_f = H.*FSK_Rx3_f ;
454
455 % The output of the low pass filter (Time domain)
456 FSK_Rx1_filtered_t = real(ifft(ifftshift(FSK_Rx1_filtered_f)) / ts);
457 FSK_Rx2_filtered_t = real(ifft(ifftshift(FSK_Rx2_filtered_f)) / ts);
458 FSK_Rx3_filtered_t = real(ifft(ifftshift(FSK_Rx3_filtered_f)) / ts);
459

```

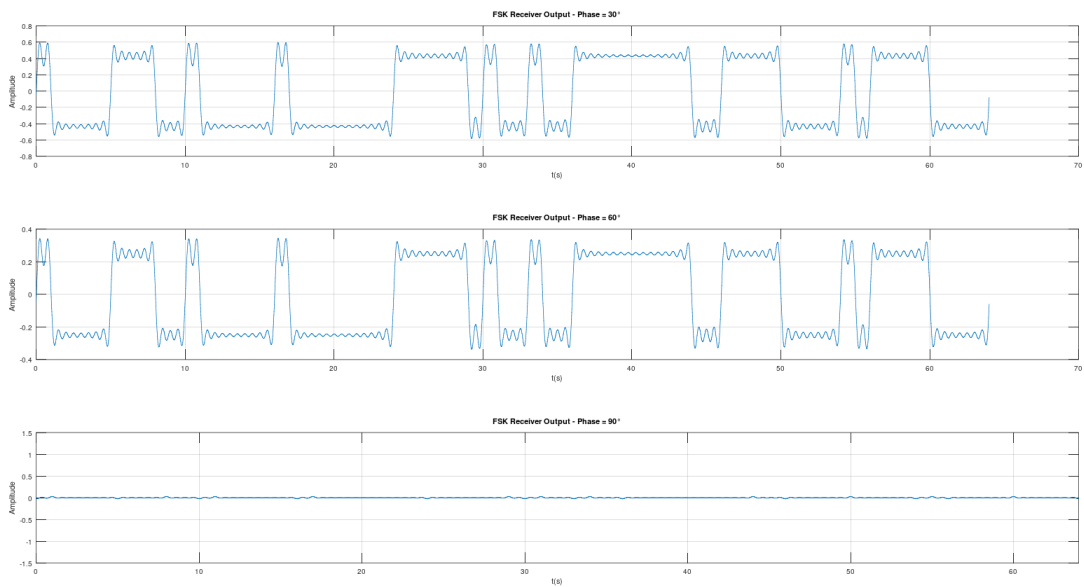
- The decision-making block code to retrieve the data

```

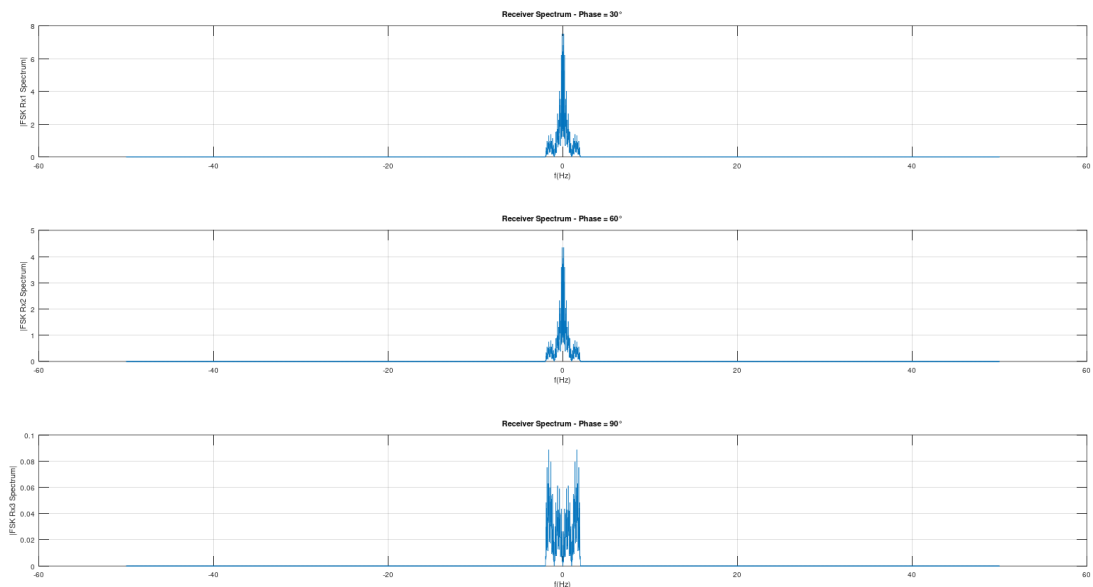
512 th = 0;
513 for i = 1:N
514     idx = (i-1)*fs + 1 : i*fs;
515     % The average value of the low-pass filter output
516     z1 = mean(FSK_Rx1_filtered_t(idx));
517     z2 = mean(FSK_Rx2_filtered_t(idx));
518     z3 = mean(FSK_Rx3_filtered_t(idx));
519
520     % Decision making
521     if z1 > th
522         decision_30(i) = 1;
523     else
524         decision_30(i) = -1;
525     end
526
527     if z2 > th
528         decision_60(i) = 1;
529     else
530         decision_60(i) = -1;
531     end
532
533     if z3 > th
534         decision_90(i) = 1;
535     else
536         decision_90(i) = -1;
537     end
538 end

```

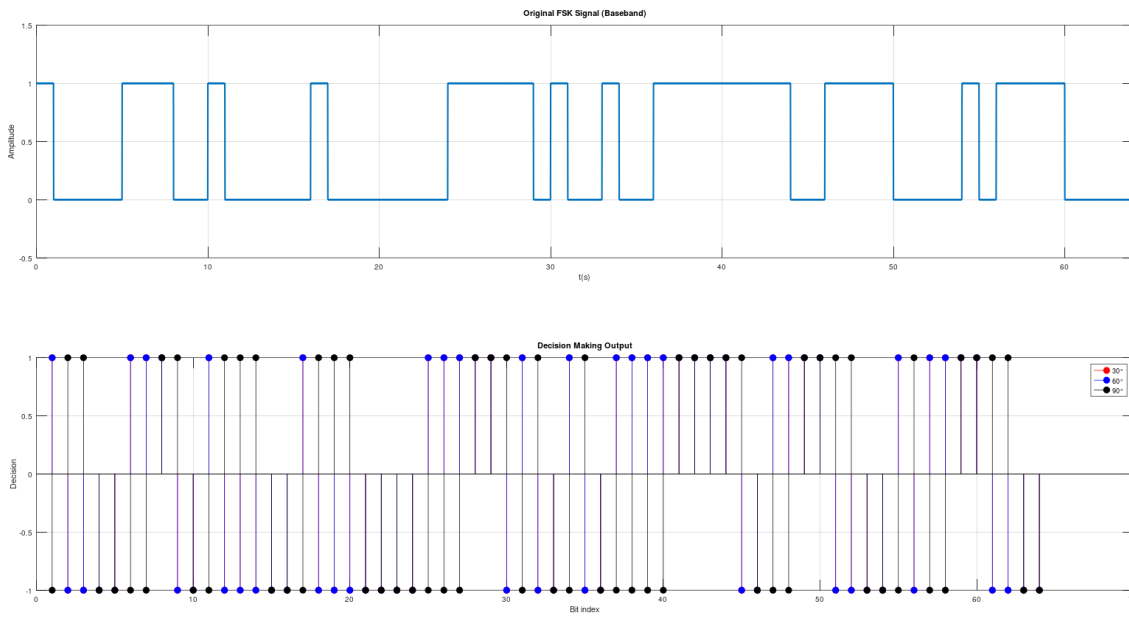
- The temporal plot of the output of the low-pass filter



- The Spectrum plot of the output of the low-pass filter



- **Comparison between the output of the decision-making device and the original baseband signal**



Notes and Comments

In FSK, the decision-making device recovers the transmitted data by comparing the received signal at the two assigned frequencies. The decision depends on which frequency is stronger, rather than on a fixed voltage threshold as in ASK or a zero threshold as in PSK. Therefore, FSK detection is less sensitive to amplitude and phase variations. However, if the receiver cannot clearly separate the two frequencies or if the filtering is not properly chosen, errors may occur in detecting the transmitted data.